

如何处理多重嵌套的工作区根目录？

rust

rust-cargo

10

如何在 Cargo 中使用多重嵌套工作区？我有以下项目结构：

```
myworkspace
├─ project_a
│   └─ Cargo.toml
├─ project_b
│   └─ Cargo.toml
│       └─ project_b_dependency
│           └─ Cargo.toml
└─ Cargo.toml
```

当我运行 `cargo build` 时，会出现错误，因为其中一个 `git` 子模块 `project_b_dependency` 是一个大型库，它自己的工作区。

这是由于存在嵌套的工作区，导致我无法成功构建。

```
$ cargo build
error: multiple workspace roots found in the same workspace:
  /myworkspace
  /myworkspace/project_b/project_b_dependency
```

有简单的解决方法吗？我想将 `project_b_dependency` 作为子模块保存在源代码控制中。

这不是与 [Refactoring to workspace structure causes extern crate imports to not work](#) 重复，因为我想知道如何处理嵌套工作区。

- Olivier

可能是 [重构工作区结构导致 extern crate 导入无法工作](#) 的重复问题。 - AbhiNickz

你希望它的行为是什么？ `cargo build --all` 命令是否应该将 `project_b_dependency` 的所有构建产物放入 `myworkspace` 目标目录中？ - Shepmaster

是的，那就是我想要的行为。谢天谢地，每个人都能回答我的问题。 - Olivier

4个回答

6

工作区不能嵌套；正如[文档](#)所述：

“一个 `crate` 只能指定 `package.workspace` 或者指定 `[workspace]` 。也就是说，一个 `crate` 不能既是工作区的根 `crate`（包含 `[workspace]` ），又是另一个工作区的成员 `crate`（包含 `package.workspace` ）。”

Cargo [工作区 RFC](#) 也明确规定了这一点：

“如果满足以下两个条件，则工作区有效：”

- 工作区只有一个根 `crate`（在 `Cargo.toml` 中带有 `[workspace]` ）。
- 所有在 `workspace.members` 中定义的工作区 `crate` 都使用 `package.workspace` 指向工作区根 `crate`。

- Ijedr

1 自从这篇文章发布以来，现在已经可以按照文档所述进行操作。此外： 一个空的[workspace]表格可以与[package]一起使用，方便地创建一个包含该包及其所有路径依赖项的工作区。 - Raphaël Duchaîne

1 我在文档中没有看到那个短语。它解释了两个示例：一个带有根包的工作区和一个具有虚拟根目录且（可能）有多个子级的工作区.....但它没有提到进一步嵌套，即这些子级是否带有自己的子级工作区。该问题在开放状态下的存在似乎证实了这一点？ <https://github.com/rust-lang/cargo/issues/5042> - Andrew Mackenzie

5

看起来嵌套工作区很难管理，因此有一个可能性是改变项目的布局：

```
.
├─ myworkspace
│   ├─ Cargo.lock
│   ├─ Cargo.toml
│   ├─ project_a
│   │   ├─ Cargo.toml
│   │   └─ src
│   │       └─ lib.rs
│   ├─ project_b
│   │   ├─ Cargo.toml
│   │   └─ src
│   │       └─ lib.rs
│   └─ src
│       └─ main.rs
└─ project_b_dependency
    ├─ Cargo.toml
    └─ src
        └─ lib.rs
```

在 myworkspace/Cargo.toml 文件中：

```
[workspace]
members= ["project_a", "project_b"]
```

在 myworkspace/project_b/Cargo.toml 文件中。

```
[dependencies]
project_b_dependency = {path = "../../project_b_dependency"}
```

我尝试使用 workspace.exclude 属性与您的布局，但没有成功。

- attdona

那是不是意味着工作区 project_b_dependency 只能一次性全部导入？还是我可以只导入其中一个crate？ - Boiethios

1

我找到了一种保持git子模块的方法。

1. 将子模块项目的工作区成员添加到父级Cargo.toml中
2. 删除子模块项目的Cargo.toml
3. 使用submodule_name/crate路径来定义依赖关系。

- jolestar

0

我找到了一种方法来解决这个问题，但它需要你引入的子模块遵循类似的模式进行合作。我的问题是，我有一个名为的库，它作为一个工作区存在于仓库中，然后有一个 **common_lib**，它是和的依赖项（虽然不是问题的具体要求，但这是我试图解决的问题）。这些代表了我们当前许多小工作区仓库的状态，都在git上。像这样的布局

```
workspace root
├─ Cargo.toml (workspace)
└─ my_app (executable)
```

```

|   └─ Cargo.toml
└─ common_lib (lib, via submodule)
|   └─ Cargo.toml
└─ bigger_lib_repo (workspace, via submodule)
    └─ Cargo.toml (workspace)
        └─ common_lib (lib, via submodule)
            └─ Cargo.toml
└─ bigger_lib (lib, not a submodule)

```

看起来像这样的内容

```

# ./Cargo.toml
[workspace]

members = ["app"]

# Important to avoid the multiple workspaces error
exclude = ["bigger_lib_repo"]

# Unsure if this is necessary
resolver = "2"

[patch.crates-io]
# The common dep gets overridden for everybody here
common_lib = { path = "../common_lib" }

# ./my_app/Cargo.toml
[package]
name = "my_app"
version = "0.1.0"
edition = "2021"

[dependencies]
# Note this isn't specced using a path
common_lib = { version = "0.1.0" }
bigger_lib = { version = "0.1.0", path = "../bigger_lib_repo/bigger_lib" }

# ./bigger_lib_repo/Cargo.toml
[workspace]

members = [ "bigger_lib" ]

resolver = "2"

[patch.crates-io]
# Same as the root workspace
common_lib = { path = "../common_lib" }

# ./bigger_lib_repo/bigger_lib/Cargo.toml
[package]
name = "bigger_lib"
version = "0.1.0"
edition = "2021"

[dependencies]
# Note this isn't specced using a path
common_lib = { version = "0.1.0" }

```

所有其他的`Cargo.toml`文件与你所期望的没有特别的的不同。有了这个模式，我可以： - 无问题地构建工作空间根目录 - 无问题地构建嵌套的工作空间根目录 - 从嵌套库中使用公开导出的包成员（例如：`bigger\_lib::common\_lib::MyStruct`）

- Andrew Lipscomb

